WhoPlay — Guia 1: Contas e perfil

Oct 28, 2026 · @Sebastião Cardoso

Responsável sugerido: Sebastião Cardoso. Este guia cobre o login, a criação de conta, o onboarding e o perfil, e ainda a base comum (servidor, ligação ao MySQL, serviço da API na app) de que os outros 3 guias precisam. Por isso é o primeiro a arrancar. O código de cada ficheiro está no guia principal, "WhoPlay — Guia da base de dados e da API"; aqui fica o que te cabe, por que ordem e como sabes que acabaste.

Guia principal (todo o código): WhoPlay — Guia da base de dados e da API

O que vais construir

No fim deste guia, uma pessoa consegue criar conta, entrar, escolher desportos e ver o seu perfil com dados verdadeiros da base de dados. Cobre os requisitos RF01 a RF08.

Camada	O que é teu
Ecrãs	01 Login, Criar conta (novo), 02 Onboarding, 03 Perfil
Tabelas	`utilizadores`, `modalidades`, `utilizador_modalidades`, `disponibilidades`
Rotas da API	`POST /auth/registo`, `POST /auth/login`, `GET /auth/eu`, `GET /modalidades`, `GET /perfil`, `PUT /perfil`, `PUT /perfil/modalidades`, `PUT /perfil/disponibilidade`
Base comum (só tu)	Projeto Node, `.env`, `config.js`, `db.js`, `erros.js`, `datas.js`, `server.js`, `middleware/auth.js`; na app, `api.dart` (parte base), `formatos.dart`, `async_view.dart`

Ficheiros

Ficheiro	Criar ou alterar	Onde está o código no guia principal
`database/schema.sql` (secções 1 a 4)	Criar	Parte 3
`database/seed.sql` (modalidades, utilizadores, desportos, disponibilidade)	Criar	Parte 4

Ficheiro	Criar ou alterar	Onde está o código no guia principal
`backend/package.json`, `.env.example`, `.env`, `.gitignore`	Criar	Parte 6.1 a 6.3
`backend/src/config.js`, `db.js`	Criar	Parte 6.4 e 6.5
`backend/src/utils/erros.js`, `datas.js`	Criar	Parte 6.6 e 6.7
`backend/src/server.js`	Criar	Parte 6.8
`backend/src/middleware/auth.js`, `routes/auth.js`	Criar	Parte 7
`backend/src/routes/modalidades.js`, `perfil.js`	Criar	Parte 8
`pubspec.yaml` (+ `http`)	Alterar	Parte 12.2
`lib/services/api.dart` (base + autenticação + perfil)	Criar	Parte 12.4
`lib/data/models.dart` (`Modalidade`, `ModalidadeNivel`, `Disponibilidade`, `Perfil`)	Criar	Parte 12.5
`lib/utils/formatos.dart`, `lib/widgets/async_view.dart`	Criar	Parte 12.6 e 12.7
`lib/screens/login_screen.dart`, `register_screen.dart`, `onboarding_screen.dart`, `profile_screen.dart`	Alterar / criar	Parte 12.8 a 12.11

Quem faz o quê na equipa

Guia	Responsável sugerido	Funcionalidade
1	Sebastião	Contas e perfil + base comum
2	Tchali	Jogadores e compatibilidade
3	Hélio	Jogos e check-in
4	Cristiano	Convites, avisos e integração final

Os nomes são uma sugestão: troquem à vontade. A única regra é o Guia 1 começar primeiro.

Antes de começar

- XAMPP com o MySQL a arrancar (guia principal, Parte 1).
- Node.js 18 ou mais recente (`node --version`).
- A app Flutter do primeiro guia a funcionar, com `flutter run -d chrome` .
- VS Code com a extensão Thunder Client.

Trabalha num ramo só teu, criado a partir do `main` atualizado:

```
git checkout main
git pull
git checkout -b feature/contas
```

Passo a passo

O trabalho divide-se em 2 fases. A **Fase 0** é a base comum e trava os colegas enquanto não estiver no `main` : façam-na numa chamada em grupo, contigo a partilhar o ecrã, e entreguem-na no mesmo dia. A **Fase 1** é a tua funcionalidade, feita em paralelo com os outros guias.

Fase 0 — Base comum (entregar primeiro)

- [ ] Criar `database/schema.sql` só com o cabeçalho (`DROP DATABASE` , `CREATE DATABASE` , `USE whoplay`) e as secções 1 a 4: `utilizadores` , `modalidades` , `utilizador_modalidades` , `disponibilidades` (Parte 3).
- [ ] Criar `database/seed.sql` com o `USE whoplay` , as modalidades, o `SET @pass` , os utilizadores, os desportos e as disponibilidades (Parte 4, até antes de "Jogo de Padel").
- [ ] Correr os 2 scripts e confirmar no phpMyAdmin que há 4 tabelas com dados.
- [ ] Criar o projeto `backend/` e instalar os pacotes (Parte 6.1 a 6.3).
- [ ] Copiar `config.js` , `db.js` , `utils/erros.js` , `utils/datas.js` e `middleware/auth.js` (Parte 6.4 a 6.7 e 7.1).
- [ ] Copiar o `server.js` (Parte 6.8) com as rotas dos colegas **comentadas**, como abaixo. Cada colega tira o `//` da sua linha quando a rota existir.
- [ ] Na app: juntar o `http` ao `pubspec.yaml` e criar `formatos.dart` e `async_view.dart` (Parte 12.2, 12.6, 12.7).

- [ ] Criar `api.dart` só com a parte base (ver abaixo) e `models.dart` só com as tuas 4 classes.
- [ ] Commit, push e pedido de merge para o `main`. Avisa a equipa no grupo: "Fase 0 no main, podem começar".

No `server.js`, as rotas protegidas ficam assim até os colegas terminarem:

```
// Rotas protegidas (precisam de token)
app.use('/perfil', autenticar, require('./routes/perfil'));
// app.use('/jogadores', autenticar, require('./routes/jogadores')); // Guia
2
// app.use('/jogos', autenticar, require('./routes/jogos'));         // Guia
3
// app.use('/convites', autenticar, require('./routes/convites'));   // Guia
4
// app.use('/avisos', autenticar, require('./routes/avisos'));       // Guia
4
```

No `api.dart`, copia da Parte 12.4 a classe `ApiException`, o `baseUrl`, o `_token`, o `terminarSessao`, o `_pedido`, o `_lista` e os blocos "autenticação" e "perfil". No lugar dos outros blocos, deixa marcadores para os colegas preencherem. Assim, cada um mexe só na sua zona do ficheiro e o Git junta tudo sem conflitos:

```
  // ------------------------------------------------------------------
jogadores (Guia 2)

  // ------------------------------------------------------------------ jogos
(Guia 3)

  // ------------------------------------------------------------------
convites e avisos (Guia 4)
}
```

Faz o mesmo no fim do `models.dart`: `// Guia 2: Criterio, Jogador`, `// Guia 3: Participante, Jogo`, `// Guia 4: Convite, Aviso`.

Fase 1 — Contas e perfil

API

- [ ] `routes/auth.js` (Parte 7.2) e `routes/modalidades.js` (Parte 8.1).
- [ ] `routes/perfil.js` (Parte 8.2). Ele usa o `NIVEIS` de `utils/compatibilidade.js`, que é do Guia 2. Até lá, troca essa linha por `const NIVEIS = ['Iniciante', 'Intermédio', 'Avançado'];`. Quando o Guia 2 entrar no `main`, voltas a prôr o `require`.

- [ ] Testar no Thunder Client os pedidos 1 a 5 da Parte 11.3 e o passo 1 da Parte 11.4 (registo).

App

- [ ] `login_screen.dart` (Parte 12.8).
- [ ] `register_screen.dart`, que é um ficheiro novo (Parte 12.9).
- [ ] `onboarding_screen.dart` (Parte 12.10), incluindo as 2 mudanças no `_SportTile`.
- [ ] `profile_screen.dart` (Parte 12.11), com o botão "Terminar sessão".
- [ ] `flutter analyze` sem problemas.

Enquanto os colegas não entregam, a app compila na mesma: os ecrãs deles continuam a usar o `mock_data.dart`. **Não apagues o `mock_data.dart`**: isso fica para o Guia 4, no fim.

Ligações com os colegas

Entregas	A quem	Quando
Tabela `utilizadores` + `db.js` + `server.js` + `auth.js` (token)	Todos	Fase 0
Tabela `modalidades` (ids 1 a 8)	Guias 2, 3 e 4 (chaves estrangeiras)	Fase 0
Tabela `disponibilidades` com dados	Guia 2 (compatibilidade)	Fase 0
`Api._pedido`, `ApiException`, `formatos.dart`, `AsyncView`	Guias 2, 3 e 4	Fase 0
`HomeShell.profileTab` a funcionar depois do Onboarding	Guia 4 (mexe no `home_shell.dart`)	Fase 1

Recebes	De quem	Enquanto não chega
`NIVEIS` em `utils/compatibilidade.js`	Guia 2	Usa a constante local (Fase 1, passo 2)

Regras para não haver conflitos no Git: no `schema.sql` e no `seed.sql`, cada guia escreve só nas suas secções; no `api.dart` e no `models.dart`, cada um escreve só debaixo do seu marcador; no `server.js`, cada um só tira o `//` da sua linha. Se precisares de mudar uma tabela tua depois da Fase 0 (uma coluna nova, por exemplo), avisa o grupo antes: os outros têm de voltar a correr os scripts.

Testes de aceitação e entrega

O teu trabalho está pronto quando tudo isto passa, com a base de dados acabada de recarregar:

- [ ] `GET /saude` → `{"api":"ok","base_de_dados":"ok"}`
- [ ] `GET /modalidades` → 8 desportos
- [ ] Login com palavra-passe errada → 401 "Email ou palavra-passe incorretos."
- [ ] `GET /perfil` sem token → 401; com o token do Sebastião → 2 desportos e 3 horários
- [ ] Registo com um email que já existe → 409
- [ ] `PUT /perfil/disponibilidade` com a hora de fim antes da de início → 400
- [ ] Na app, o login com sebastiao@email.com abre o separador Jogadores
- [ ] Na app, Criar conta → escolher 2 desportos → Continuar abre o Perfil com esses 2 desportos, em Iniciante
- [ ] Perfil → Terminar sessão volta ao Login
- [ ] `flutter analyze` sem problemas

Entrega:

```
git add database backend lib pubspec.yaml pubspec.lock
git commit -m "Contas e perfil: BD, API e ecrãs"
git push -u origin feature/contas
```

No GitHub, abre um Pull Request de `feature/contas` para `main` e pede a um colega que o reveja. Confirma antes que o `.env` e a `node_modules/` **não** estão no commit (`git status`).